

Research Plan for LLM-Based Trading Agents in Continuous Double-Auction Markets

Background and Related Work

Continuous Double Auctions (CDAs) – where buyers and sellers submit orders continuously to a central order book – are the backbone of real-world equity and commodity markets. A variety of open-source simulators exist for CDA market research. For example, the **Bristol Stock Exchange (BSE)** is a public-domain simulator of CDA markets. Rollins & Cliff showed that extending BSE to a threaded version (TBSE) – which runs agents in parallel – significantly alters which trading algorithms perform best. Modern high-fidelity platforms include **ABIDES** (an agent-based, NASDAQ-protocol emulator) and **PyMarketSim** (a Python environment optimized for deep-RL in markets). These frameworks support large numbers of agents, realistic limit-order-book matching, and configurable latencies.

Recent literature has begun exploring Large Language Models (LLMs) as trading agents in such CDA simulations. Lopez-Lira (2025) built an open-source CDA simulator where GPT-4-based agents – instructed as *value investors*, *momentum traders*, *market makers*, etc. – place orders via structured JSON output. He finds that LLMs can follow strategy instructions accurately and generate realistic market behavior (price discovery, bubbles, underreaction). Agrawal et al. (2025) simulate LLM **seller agents** in a CDA and show that allowing inter-agent communication dramatically increases collusion. Xiao et al. (2024) propose *TradingAgents*, a multi-LLM framework modeling a trading firm: separate LLMs act as fundamental/sentiment/technical analysts, bullish/bearish researchers, traders, and risk managers. Their experiments (on stock-return data) show this role-based LLM system outperforms baselines in returns and Sharpe ratio. Other works (FinMem, FinCon) integrate memory modules or self-reflection into LLM trading agents for improved performance on portfolio tasks.

These studies indicate the promise of LLM agents, but much remains unexplored. In particular, different model variants (LLama-3, Claude, open-source LLMs), fine-tuning or RL training of LLMs, richer market information, and complex multi-agent interactions (e.g. hierarchical roles, collusion scenarios) have not been fully tested. We will leverage and

extend these ideas: combining structured prompts, memory, and communication in new ways, and evaluating them against traditional algorithms in CDA settings.

Simulation Platforms, Data and Benchmarks

We will conduct experiments in realistic CDA market simulators. As a baseline, we can use the **BSE/TBSE** simulators with classical agents (e.g. Gjerstad–Dickhaut, ZIP, GD, zero-intelligence). For larger-scale or RL-enabled runs, we will use frameworks like **ABIDES-Gym** or **PyMarketSim**. These support thousands of agents, detailed matching engines, and API interfaces for agent actions. PyMarketSim in particular is designed for fast multi-agent RL with continuous double-auction matching and private valuations.

Our market scenarios will include a single stock (or a small portfolio) with an underlying *fundamental value* process (e.g. mean-reverting dividend stream) and stochastic news shocks. We will also assemble multi-modal market data to feed our LLM agents: for instance, historical prices, corporate fundamentals (via Yahoo Finance), financial news headlines (Bloomberg/Reuters feeds), and social sentiment (Twitter/Reddit). This mirrors Xiao et al.’s setup where LLM analysts process multi-source data. By contrast, classical agent benchmarks will have only price and fundamental information. We will calibrate all simulations so that classical “benchmark” traders achieve reasonable efficiency, enabling fair comparison to LLM agents. Key benchmarks include profit attained by Zero-Intelligence (ZI) traders versus LLM agents, and stability of the price around fundamental value (price discovery).

*Figure: A TradingAgents-style multi-LLM architecture. Separate LLMs act as **Analysts** (fundamental, sentiment, technical, etc.), **Researchers** (bullish/bearish debate), **Traders**, and **Risk Managers**, collaborating to make trading decisions.*

Agent Design: LLM Models and Prompts

We will implement **heterogeneous trading agents** following the literature. Each LLM-based agent will use a system prompt defining its role (e.g. “You are a speculator looking for market inefficiencies” or “You are a value investor using dividend discount models”). Agents output structured orders (JSON with fields like “action: buy/sell, quantity, price”) along with

natural-language reasoning. This two-tier output (machine-readable + human-readable reasoning) was effective in prior work.

Roles to implement include *value investors* (using fundamentals), *momentum traders*, *market-makers* (placing bids/asks around mid-price), and *noise traders*. We will also test **complex architectures** inspired by TradingAgents: e.g. a group of LLM **Analyst agents** that share findings with a **Trader** agent, overseen by a **Risk Manager**. To facilitate this, agents might exchange summaries (e.g. bullish vs bearish notes, as in) or use shared memory buffers (akin to FinMem’s memory module).

Crucially, we will experiment with *model variations* not yet covered in literature. Previous work mainly used GPT-4 (Turbo/4o). We will also test open models (LLaMA-3, GPT-3.5, Claude) to see how model choice affects strategy. As Agrawal et al. note, collusive tendencies varied by model. We will tune parameters like temperature and system instructions, and possibly fine-tune or RL-train LLMs. For example, one experiment could reward an LLM via policy gradients for profitable orders, blending chain-of-thought with learned policy. These RL or fine-tuning extensions are largely unexplored in prior CDA/LLM studies.

Experimental Setup and Metrics

Our experiments will run **market simulations** in discrete time (e.g. trading “rounds” or ticks). In each round, agents observe the current order book, last price(s), and any external signals (news). They then issue buy/sell orders. The exchange matches orders continuously by price-time priority. We will record trade history, order-book snapshots, and each agent’s balance/holdings.

We will compare several conditions, for example:

- **Baseline agents only:** Traditional CDA algorithms (ZIP, GD, ZI).
- **LLM-only:** All traders are LLM agents following specified strategies.
- **Mixed:** LLM agents trading alongside traditional agents.
- **Communication enabled/disabled:** To test collusion, we allow some LLM sellers to chat via encrypted messages (like Agrawal et al.) vs. no communication.
- **Role structures:** Single-agent versus hierarchical setups (e.g. TradingAgents pipeline).

We will evaluate performance with standard financial metrics. Primary metrics include **profit and loss**, **Cumulative Return (CR)**, **Sharpe Ratio (SR)**, and **Maximum Drawdown (MDD)**.

These capture absolute returns and risk-adjusted returns. We will also analyze market-level outcomes: **price efficiency** (convergence to fundamental value), bid-ask **spread and depth**, volatility, and **liquidity**. Prior work measures like “price discovery” and occurrence of bubbles will be checked. For collusion tests, we will measure how consistently sellers fix prices or divide market share. Finally, we will inspect agent reasoning outputs qualitatively to verify strategy adherence.

Implementation Plan and Tools

Programming: All experiments will be implemented in Python. We will use LLM APIs such as OpenAI’s GPT (for GPT-4/Turbo/4o) and HuggingFace models (for open LLMs) to generate agent actions. Prompts and responses will be handled via function calls (as in OpenAI’s API) to get structured JSON order decisions.

Market simulator: We will adapt existing codebases: for example, **BSE (Python)** for fast prototyping of simple experiments, and **PyMarketSim** or **ABIDES** for large-scale tests. If using ABIDES, its Gym wrappers allow RL integration. The PyMarketSim framework is already designed for continuous markets with private values. We will implement order submission and clearing logic consistent with both simulators.

Agents and prompts: We will start by coding baseline rule-based traders (ZI, zero intelligence with new strategy, and others from BSE). Then we will script LLM agents: each agent loop will feed the market state and relevant memory/observations into the LLM prompt (via system and user messages), and parse the returned JSON order. We will design structured system messages based on [6] and [22] to ensure clarity. For example:

- **Value investor prompt:** includes the current fundamental price and news summary, instructing the agent to buy if price < value, sell if >.
- **Momentum trader prompt:** includes recent price trend and asks the agent to follow momentum.
- **Market-maker prompt:** instructs placing bids/asks around mid-price to collect spread.

We will add a **memory buffer** for each agent to store past outcomes (inspired by FinMem and FinCon). This memory can be included in prompts to give context (e.g. “Last round you bought at \$X, now the price is Y”).

Reproducibility: Experiments will be scripted to allow reproducible runs. We will fix random seeds for stochastic elements, log all prompts and responses, and version-control the code. We will use standard Python libraries (NumPy/Pandas for data, Matplotlib for charts) and possibly RL libraries (Stable Baselines3 or RLLib) if we train LLM policies.

Baseline Comparison and Reporting

We will compare LLM agents against classical strategies under identical market conditions. For example, if a “value strategy” with LLM returns X, we will compare it to a simple algorithmic value trader (e.g. limit order around fundamental) to assess the LLM’s added value. Results (profits, SR, price series) will be compiled into tables and plots.

All experimental conditions, prompts, and parameters will be documented. We will package code and data in a public repository (with instructions) to ensure reproducibility. Our final report will include: problem motivation, detailed methodology (platform, agent design, prompts), results with analysis, and discussion of insights (e.g. which LLM models or agent roles performed best, and why).

References: We will cite all external sources as shown. Key references include Lopez-Lira (2025) on LLM traders in CDA, Agrawal et al. (2025) on LLM collusion, and the TradingAgents framework. We will also reference market simulators (BSE/TBSE, ABIDES, PyMarketSim) and evaluation standards (Sharpe, drawdown) throughout. These citations are integrated above to credit prior work and justify our choices.